

Linear Regression: The Normal Equations, Gradient Methods, and Why the Closed Form Is Not Always the Right Tool

Yuval Lavie

August 10, 2026

Abstract

Least squares is the rare model with a complete theoretical solution — the normal equations — and the standard testbed for gradient methods. We derive the closed form, reach the same optimum by GD and SGD (with the autograd identity check of the Bernoulli and Gaussian notes), and then make the practical point: even for this linearly solvable problem there are four independent reasons to prefer iterative methods, one of which (conditioning) the companion script demonstrates numerically.

1 Setup

Data $(x_i, y_i)_{i=1}^n$ with $x_i \in \mathbb{R}^d$ (first coordinate fixed to 1 for the intercept), stacked as $X \in \mathbb{R}^{n \times d}$ and $y \in \mathbb{R}^n$. Least squares minimizes

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|^2. \quad (1)$$

This is also maximum likelihood. Under the model $y_i = x_i^\top \theta + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, s^2)$ i.i.d., the NLL of the sample is $\frac{1}{2} \log(2\pi s^2) + \frac{1}{2s^2n} \sum_i (y_i - x_i^\top \theta)^2$: up to constants not involving θ , exactly (1). Least squares is the Gaussian-noise instance of the MLE program of the previous notes.

2 Route 1: the theoretical solution

Proposition 1. *If X has full column rank, the unique minimizer of (1) is*

$$\boxed{\hat{\theta} = (X^\top X)^{-1} X^\top y} \quad (2)$$

Proof. The gradient of (1) is

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y). \quad (3)$$

Setting it to zero gives the *normal equations* $X^\top X \theta = X^\top y$. The Hessian $\frac{1}{n} X^\top X$ is positive definite when X has full column rank (for $u \neq 0$, $u^\top X^\top X u = \|Xu\|^2 > 0$), so J is strictly convex and the solution of the linear system is its unique minimizer. $\square$

Geometrically, (2) makes the residual orthogonal to the column space of X : $X^\top(y - X\hat{\theta}) = 0$ — $\hat{y} = X\hat{\theta}$ is the orthogonal projection of y onto $\text{col}(X)$.

3 Routes 2–4: gradient methods

No reparameterization is needed here — θ is already unconstrained, and (1) is smooth and convex.

GD iterates the full gradient (3): $\theta \leftarrow \theta - \frac{\eta}{n} X^\top(X\theta - y)$, cost $O(nd)$ per step, converging linearly (the objective is strongly convex when $X^\top X \succ 0$).

SGD replaces (3) with the one-sample estimate

$$g_i(\theta) = (x_i^\top \theta - y_i) x_i, \quad \mathbb{E}_i[g_i] = \nabla J(\theta), \quad (4)$$

cost $O(d)$ per step: the residual on one point, pushed along that point’s features. As always, a constant step leaves a noise ball; the script Polyak-averages the last pass. Mini-batches interpolate.

Autograd. The script writes the per-sample loss $\frac{1}{2}(x_i^\top \theta - y_i)^2$ in torch, drives routes 3 and 4 through the identical sample schedule, and asserts the trajectories coincide — `backward()` must produce exactly (4).

4 Why not always use the closed form?

Four independent reasons, in increasing order of importance:

(a) Cost and memory. Forming $X^\top X$ costs $O(nd^2)$ and solving costs $O(d^3)$, with $O(d^2)$ memory. At $d \sim 10^5$ (text features, genomics, one-hot blowups) that is beyond reach, while an SGD step stays $O(d)$.

(b) Conditioning. Solving the normal equations works with $X^\top X$, and

$$\kappa(X^\top X) = \kappa(X)^2 : \quad (5)$$

forming the Gram matrix *squares* the condition number, and roughly $\log_{10} \kappa$ decimal digits are lost to rounding. The script builds two nearly collinear features ($\kappa(X) \sim 10^5$, so $\kappa(X^\top X) \sim 10^{10}$) and solves in `float32`: the normal-equations solution loses essentially all accuracy, while QR-based `lstsq` — which orthogonalizes X directly and pays only $\kappa(X)$ — stays correct to several digits. Moral: even when you do want the exact solution, you want it via QR/SVD, not via $(X^\top X)^{-1}$.

(c) Streaming and online data. SGD consumes samples as they arrive and never materializes the dataset; the closed form needs all of X at once. (A middle ground exists — recursive least squares — but it maintains a $d \times d$ state, inheriting problem (a).)

(d) Generality. The closed form is a property of *this loss*, not a method: add a sigmoid, a hinge, or a hidden layer and it disappears, while the gradient recipe — compose, differentiate, descend — survives unchanged. Practicing it on the one model where we can check every step against (2) is the point of this note.

5 Experiment

With $\theta^* = (2, 3, -5)$, $n = 2000$, unit noise (seed 7): the normal equations, GD, and Polyk-averaged SGD agree to a few 10^{-2} (SGD's noise floor); hand and autograd SGD trajectories coincide to 10^{-10} . The conditioning demo prints the two `float32` errors against a `float64` reference — normal equations off by $O(1)$, QR off by $O(10^{-4})$ — realizing (5) in twelve lines of numpy.